



AMRITA
VISHWA VIDYAPEETHAM

DSA
LAB MANUAL

Submitted to	
Name	Dr. Rajkumar Batchu
Department	CSE
Designation	Assistant Professor
Submitted by	
Name	Y.VAMSI
Roll No	AV.SC.U4CSE25154
Year/Sem/Section	2 nd year / Sem-3 / CSE-B

INDEX

LAB	QUESTIONS	PGNO	REMARKS
1	a) A rocket launch system displays a countdown before launch. Instead using loop implement the countdown from n to 1, followed by "Launch!". b) Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n (power) where, P = principle growth factor, n = number of years. c) An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. d) In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. e) Write a recursive python program to print the first n terms of fibonacci series.	4-8	
2	a)Program to implement LINEAR SEARCH b)Program to implement BINARY SEARCH (1) With Sorted List: (2) Without Sorted List:	9-11	
3	a)Write a Program for BUBBLE SORT b)Write a program for INSERTION SORT c)Write a program for SELECTION SORT;	12-14	
4	a)Write a Program for Quick SORT	15-17	

	b)Write a Program for MERG SORT:		
5	a)Write a Program to implement single Linked List: b)Write a Program to implement Double Linked List: c)write a program to implement circular Linked List:	18-26	
6	a)Write python program to implement stack using Array b)Write python program to implement stack using LinkedList	26-31	

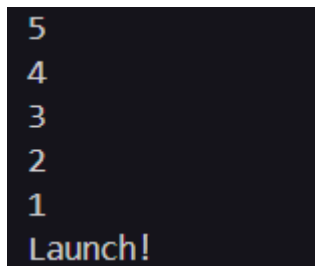
LAB-1

a) A rocket launch system displays a countdown before launch. Instead using loop implement the countdown from n to 1, followed by "Launch!".

CODE:

```
1. def countdown(n):  
2.     if n <= 0:  
3.         print("Launch!")  
4.     else:  
5.         print(n)  
6.         countdown(n - 1)  
7.  
8. countdown(5)
```

OUTPUT:



```
5  
4  
3  
2  
1  
Launch!
```

EXPLANATION:

Base Case: When the number n reaches 0 or less, the recursion stops and the program prints "Launch!".

Recursive Step: If n is greater than 0, the program prints the current number and calls itself with n - 1, moving one step closer to the base case.

b) Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n (power) where, P = principle growth factor, n = number of years.

CODE:

```
1. def calculate_power(P, n):
2.     if n == 0:
3.         return 1
4.     else:
5.         return P * calculate_power(P, n - 1)
6.
7. print(calculate_power(20, 5))
```

OUTPUT:

3200000

EXPLANATION:

Base Case: Any number raised to the power of 0 is 1. When n reaches 0, the function returns 1 and stops recurring.

Recursive Step: The function multiplies the base P by the result of the same function called with $n - 1$, effectively chaining the multiplication n times.

c) An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

CODE:

```
1. def search_employee(emp_list, emp_id, index=0):
2.     if index == len(emp_list):
3.         return False
4.     if emp_list[index] == emp_id:
5.         return True
6.     return search_employee(emp_list, emp_id, index + 1)
7.
8. employees = [101, 105, 108, 112]
9. print(search_employee(employees, 108))
```

OUTPUT:

True

EXPLANATION:

Base Cases: The function stops and returns False if the index goes out of bounds (meaning the entire list was searched). It returns True if the item at the current index matches the emp_id.

Recursive Step: If the current item is not a match, the function calls itself while incrementing the index by 1 to check the next element in the list.

ERROR TABLE:

SNO	ERROR	SOLUTION
1	Line 2: if index len(emp_list): is missing the equality operator	Change to if index == len(emp_list):

d) In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula ($n!$). Develop a recursive program to calculate the factorial of a given number.

CODE:

```
1. def calculate_factorial(n):
2.     if n == 0 or n == 1:
3.         return 1
4.     else:
5.         return n * calculate_factorial(n - 1)
6.
7. print(calculate_factorial(5))
```

OUTPUT:

120

EXPLANATION:

Base Case: By mathematical definition, the factorial of 0 or 1 is 1. When n hits either of these, the recursion halts and returns 1.

Recursive Step: The function multiplies the current integer n by the factorial of $n - 1$, evaluating to $n \times (n - 1) \times (n - 2) \dots \times 1$ as it resolves back up the call stack.

e) Write a recursive python program to print the first n terms of fibonacci series.

CODE:

```
1. def fibonacci(n):
2.     if n <= 0:
3.         return 0
4.     elif n == 1:
5.         return 1
6.     else:
7.         return fibonacci(n - 1) + fibonacci(n - 2)
8.
9. def print_fibonacci_series(terms):
10.     for i in range(terms):
11.         print(fibonacci(i), end=" ")
12.     print()
13.
14. print_fibonacci_series(7)
```

OUTPUT:

0 1 1 2 3 5 8

EXPLANATION:

Base Cases: The 0th term of the sequence is 0, and the 1st term is 1. The function directly returns these values when n is 0 or 1.

Recursive Step: For any position greater than 1, the function calculates the value by calling itself twice to add the two preceding terms together (n - 1 and n - 2).

LAB-2

a)Program to implement LINEAR SEARCH

CODE

```
1. def linear_search(arr, target):
2.     for i in range(len(arr)):
3.         if arr[i] == target:
4.             return i
5.     return -1
6.
7. my_array = [10, 23, 45, 70, 11, 15]
8. target_val = 70
9. print(linear_search(my_array, target_val))
```

OUTPUT:

3

EXPLANATION:

How it works: The algorithm sequentially checks each element in the array from the beginning (index 0) to the end.

Condition: During each iteration, it compares the current element to the target.

Result: If a match is found, it immediately returns the index of that element. If the loop completes and the element is never found, it returns -1 to indicate the target is not in the array.

Time Complexity: $O(n)$ because in the worst-case scenario, it has to check every single element.

b) Program to implement BINARY SEARCH

(1) With Sorted List:

CODE:

```
1. def binary_search_sorted(arr, target):
2.     low = 0
3.     high = len(arr) - 1
4.
5.     while low <= high:
6.         mid = (low + high) // 2
7.         if arr[mid] == target:
8.             return mid
9.         elif arr[mid] < target:
10.            low = mid + 1
11.        else:
12.            high = mid - 1
13.
14.    return -1
15.
16. sorted_array = [11, 22, 34, 45, 55, 67, 89]
17. target_val = 45
18. print(binary_search_sorted(sorted_array, target_val))
```

OUTPUT:

3

EXPLANATION:

How it works: Binary search uses a divide-and-conquer approach but strictly requires the array to be sorted beforehand. It maintains a low and a high pointer to track the search space.

Condition: It calculates the mid point of the current search space.

- If the mid element is the target, it returns the mid index.
- If the target is strictly greater than the mid element, it discards the left half by moving the low pointer to mid + 1.
- If the target is strictly less than the mid element, it discards the right half by moving the high pointer to mid - 1.

Result: The loop continues narrowing the search space until low becomes greater than high. If the target isn't found by then, it returns -1.

Time Complexity: $O(\log n)$ because the search space is cut in half during every step.

(2) Without Sorted List:

CODE:

```
1. def binary_search_unsorted(arr, target):
2.     arr.sort()
3.
4.     low = 0
5.     high = len(arr) - 1
6.
7.     while low <= high:
8.         mid = (low + high) // 2
9.         if arr[mid] == target:
10.            return mid
11.         elif arr[mid] < target:
12.             low = mid + 1
13.         else:
14.             high = mid - 1
15.
16.     return -1
17.
18. unsorted_array = [67, 11, 89, 22, 55, 45, 34]
19. target_val = 45
20. print(binary_search_unsorted(unsorted_array, target_val))
```

OUTPUT:

3

EXPLANATION:

How it works: Because binary search relies on the data being in a specific order to eliminate halves of the array, it cannot function on unsorted data.

Condition: The code first calls arr.sort() to organize the array in ascending order.

Result: Once the array is sorted, it executes the exact same binary search logic as above.

(Note: Because the array is modified and sorted before searching, the index returned corresponds to the target's position in the newly sorted array, not its original position in the unsorted input).

Time Complexity: $O(n \log n)$ due to the initial sorting step, which dominates the $O(\log n)$ search time.

ERROR TABLE:

SNO	ERROR	SOLUTION
1.	Line 2 (Sorted): low = 6 incorrectly sets the starting index.	Change to low = 0
2.	Lines 6 & 8: mid (low + high) // 2 is missing the assignment operator.	Change to mid = (low + high) // 2
3.	Lines 12 & 14: high = mid 1 is missing a subtraction operator.	Change to high = mid - 1

WEEK-3

a)Write a Program for BUBBLE SORT

CODE:

```
1. def bubble_sort(arr):
2.     n = len(arr)
3.     for i in range(n):
4.         swapped = False
5.         for j in range(0, n - i - 1):
6.             if arr[j] > arr[j + 1]:
7.                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
8.                 swapped = True
9.         if not swapped:
10.            break
11.    return arr
```

OUTPUT:

[11, 12, 22, 25, 34, 64, 90]

EXPLANATION;

Bubble sort works by repeatedly swapping adjacent elements if they are in the wrong order. With each full pass, the largest unsorted element "bubbles" up to its correct position at the end of the array.

- **The Outer Loop (for i in range(n)):** This tracks the number of passes. After the first pass, the largest element is guaranteed to be at the very end. After the second pass, the second largest is in place, and so on.
- **The Inner Loop (for j in range(0, n - i - 1)):** This loop performs the actual comparisons. It stops at n - i - 1 because we don't need to check the last i elements—they are already in their sorted, final positions from previous passes.
- **The Swap:** if arr[j] > arr[j + 1]: checks if the left element is larger than the right element. If so, it swaps them.
- **The Optimization (swapped = True/False):** If the array is already sorted, making full passes is a waste of time. The swapped variable tracks if any swaps happened during a pass. If a full pass happens with zero swaps, the loop breaks early, saving processing time.

b)Write a program for INSERTION SORT

CODE:

```
1. def insertion_sort(arr):
2.     for i in range(1, len(arr)):
3.         key = arr[i]
4.         j = i - 1
5.         while j >= 0 and key < arr[j]:
6.             arr[j + 1] = arr[j]
7.             j -= 1
8.         arr[j + 1] = key
9.     return arr
10.
11. my_array = [12, 11, 13, 5, 6]
12. print(insertion_sort(my_array))
```

OUTPUT:

[5, 6, 11, 12, 13]

EXPLANATION:

Insertion sort builds the final sorted array one item at a time. It works exactly like sorting a hand of playing cards: you take an unsorted card, figure out where it belongs in the cards you've already sorted, and insert it there.

- **The Outer Loop (for i in range(1, len(arr))):** We start at index 1 (the second element) because we assume a single element at index 0 is already "sorted". We pull this current element out and call it the key.
- **The while Loop:** We look at the elements to the left of the key (which are already sorted). As long as we haven't reached the beginning of the array ($j \geq 0$) and the sorted element we are looking at is larger than our key ($\text{key} < \text{arr}[j]$), we shift that sorted element one space to the right ($\text{arr}[j + 1] = \text{arr}[j]$).
- **The Insertion ($\text{arr}[j + 1] = \text{key}$):** Once we find an element smaller than the key (or reach the start of the array), the while loop stops. We then drop the key into the empty space we created.

c)Write a program for SELECTION SORT;

CODE:

```
1. def selection_sort(arr):
2.     for i in range(len(arr)):
3.         min_idx = i
4.         for j in range(i + 1, len(arr)):
5.             if arr[j] < arr[min_idx]:
6.                 min_idx = j
7.         arr[i], arr[min_idx] = arr[min_idx], arr[i]
8.     return arr
9.
10. my_array = [64, 25, 12, 22, 11]
11. print(selection_sort(my_array))
```

OUTPUT:

[11, 12, 22, 25, 64]

EXPLANATION:

Selection sort works by dividing the array into two parts: a sorted portion at the front and an unsorted portion at the back. It repeatedly searches the unsorted portion to find the smallest element, and moves it to the end of the sorted portion.

- **The Outer Loop (for i in range(len(arr))):** This loop represents the boundary between the sorted and unsorted portions. The element at index i is where the next smallest number needs to go. We temporarily assume the element at i is the minimum (min_idx = i).
- **The Inner Loop (for j in range(i + 1, len(arr))):** This loop scans the rest of the unsorted array (everything to the right of i) to see if there is an element smaller than our current assumed minimum.
- **Updating the Minimum (if arr[j] < arr[min_idx]):** If we find a smaller element, we update min_idx to remember its location.
- **The Swap:** After checking the entire unsorted portion, we swap the element currently at position i with the actual smallest element we found at min_idx.

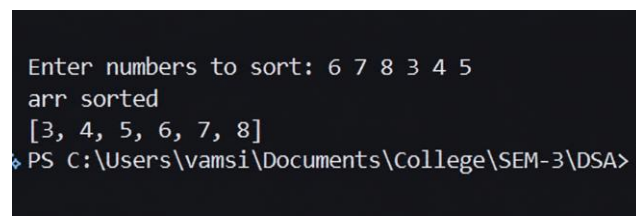
WEEK-4

a) Write a Program for Quick SORT

CODE:

```
1. def quicksort(arr, low, high):
2.     if low < high:
3.         i = low
4.         j = high
5.         pivot = low
6.
7.         while i < j:
8.             while (i < len(arr) and arr[i] <= arr[pivot]):
9.                 i += 1
10.
11.            while arr[j] > arr[pivot]:
12.                j -= 1
13.
14.            if i < j:
15.                arr[i], arr[j] = arr[j], arr[i]
16.
17.        arr[j], arr[pivot] = arr[pivot], arr[j]
18.
19.        quicksort(a, low, j-1)
20.        quicksort(a, j+1, high)
21.
22. a = list(map(int, input("Enter numbers to sort: ").split(" ")))
23. n = len(a)
24. quicksort(a, 0, n-1)
25. print("arr sorted")
26. print(a)
27.
```

OUTPUT:



```
Enter numbers to sort: 6 7 8 3 4 5
arr sorted
[3, 4, 5, 6, 7, 8]
PS C:\Users\vamsi\Documents\College\SEM-3\DSA>
```

EXPLANATION:

Algorithm Paradigm: Divide and Conquer

Pivot Selection: First element of the sub-array (`arr[low]`)

Time Complexity:

- **Best / Average Case:** $O(n \log n)$
- **Worst Case:** $O(n^2)$ (when the array is already sorted or reverse sorted)

Space Complexity: $O(\log n)$ auxiliary stack space for recursion.

b)Write a Program for MERG SORT:

CODE:

```
1. def merge_sort(arr):
2.     if len(arr) > 1:
3.         mid = len(arr) // 2
4.         left_half = arr[:mid]
5.         right_half = arr[mid:]
6.
7.         merge_sort(left_half)
8.         merge_sort(right_half)
9.
10.        i = j = k = 0
11.
12.        while i < len(left_half) and j < len(right_half):
13.            if left_half[i] < right_half[j]:
14.                arr[k] = left_half[i]
15.                i += 1
16.            else:
17.                arr[k] = right_half[j]
18.                j += 1
19.                k += 1
20.
21.        while i < len(left_half):
22.            arr[k] = left_half[i]
23.            i += 1
24.            k += 1
25.
26.        while j < len(right_half):
27.            arr[k] = right_half[j]
28.            j += 1
29.            k += 1
30.
31.        return arr
32.
33. user_input = input("Enter numbers to sort separated by spaces: ")
34. numbers = [int(x) for x in user_input.split()]
35. merge_sort(numbers)
36. print("arr sorted:")
37. print(numbers)
38.
```

OUTPUT:

```
Enter numbers to sort separated by spaces: 3 4 6 7 4
arr sorted:
[3, 4, 4, 6, 7]
PS C:\Users\vamsi\Documents\College\SEM-3\DSA> █
```

EXPLANATION:

Algorithm Paradigm: Divide and Conquer

Merging Strategy: Out-of-place merge (requires additional memory allocation for sub-arrays)

Time Complexity:

- **Best, Average, and Worst Case:** $O(n \log n)$. The array is consistently halved ($\log n$ divisions), and the merging process takes linear time (n).

Space Complexity: $O(n)$ auxiliary space is required to store the temporary left_half and right_half arrays during the merging process.

Stability: Stable. Merge sort preserves the relative order of duplicate elements because it strictly checks `left_half[i] < right_half[j]` (elements from the left take precedence if they are equal).

ERROR TABLE:

SNO	ERROR	SOLUTION
1.	Unhandled ValueError (Runtime Input)	<code>[int(x) for x in user_input.split()]</code> assumes perfect user behavior. If a user types letters, symbols, or floats (e.g., 5 2 a 9), the program immediately crashes. Wrap the input parsing step in a <code>try...except ValueError:</code> block to prompt the user again.
2.	Algorithm Logic	No Errors Occurred. The recursive division, list slicing, and pointer logic (<code>i, j, k</code>) for the merge step are perfectly executed.

WEEK-5

a)Write a Program to implement single Linked List:

CODE:

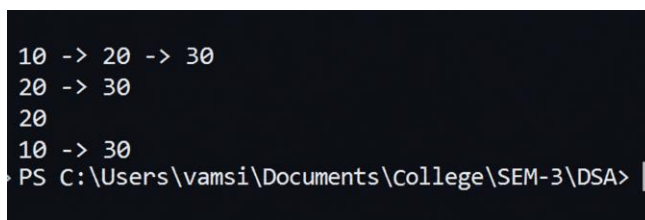
```
1. class Node:
2.     def __init__(self, data):
3.         self.data = data
4.         self.next = None
5.
6. class LinkedList:
7.     def __init__(self):
8.         self.head = None
9.
10.    def insert_at_start(self, data):
11.        new_node = Node(data)
12.        new_node.next = self.head
13.        self.head = new_node
14.
15.    def insert_at_end(self, data):
16.        new_node = Node(data)
17.        if not self.head:
18.            self.head = new_node
19.            return
20.        last = self.head
21.        while last.next:
22.            last = last.next
23.        last.next = new_node
24.
25.    def insert_at_index(self, index, data):
26.        if index == 0:
27.            self.insert_at_start(data)
28.            return
29.
30.        new_node = Node(data)
31.        current = self.head
32.
33.        for _ in range(index - 1):
34.            if not current:
35.                raise IndexError("Index out of bounds")
36.            current = current.next
37.
38.        if not current:
39.            raise IndexError("Index out of bounds")
40.
41.        new_node.next = current.next
42.        current.next = new_node
43.
44.    def delete_at_start(self):
45.        if not self.head:
46.            print("List is empty.")
47.            return
48.        self.head = self.head.next
49.
50.    def delete_at_end(self):
51.        if not self.head:
52.            print("List is empty.")
53.            return
54.        if not self.head.next:
55.            self.head = None
56.            return
57.
58.        current = self.head
59.        while current.next.next:
60.            current = current.next
61.        current.next = None
62.
63.    def delete_at_index(self, index):
64.        if not self.head:
65.            print("List is empty.")
66.            return
```

```

67.     if index == 0:
68.         self.delete_at_start()
69.         return
70.
71.     current = self.head
72.     for _ in range(index - 1):
73.         if not current.next:
74.             raise IndexError("Index out of bounds")
75.             current = current.next
76.
77.     if not current.next:
78.         raise IndexError("Index out of bounds")
79.
80.     current.next = current.next.next
81.
82. def display(self):
83.     elements = []
84.     current = self.head
85.     while current:
86.         elements.append(str(current.data))
87.         current = current.next
88.     print("-> ".join(elements) if elements else "Empty List")
89.
90. if __name__ == "__main__":
91.     ll = LinkedList()
92.
93.     ll.insert_at_start(10)
94.     ll.insert_at_end(30)
95.     ll.insert_at_index(1, 20)
96.     ll.display()
97.
98.     ll.delete_at_start()
99.     ll.display()
100.
101.     ll.delete_at_end()
102.     ll.display()
103.
104.     ll.insert_at_start(10)
105.     ll.insert_at_end(30)
106.     ll.delete_at_index(1)
107.     ll.display()
108.

```

OUTPUT:



```

10 -> 20 -> 30
20 -> 30
20
10 -> 30
PS C:\Users\vamsi\Documents\College\SEM-3\DSA>

```

EXPLANATION:

insert_at_start(data): Creates a new node, points its next reference to the current head, and updates the head to the new node.

insert_at_end(data): Traverses the list to find the last node (where next is None) and updates its reference to point to the new node. Handles empty lists by setting the head directly.

insert_at_index(index, data): Traverses to the node right before the target index (index - 1). Points the new node to the current node's next, and updates the current node to point to the new node.

delete_at_start(): Reassigns the head pointer to the second node in the list (head.next), allowing Python's garbage collector to remove the old head.

delete_at_end(): Traverses the list looking one step ahead (current.next.next). Once it finds the second-to-last node, it sets its next pointer to None.

delete_at_index(index): Traverses to the node immediately preceding the target index. It then bypasses the target node by updating the reference to current.next.next.

Data Structure Type: Linear, Dynamic (Non-contiguous memory allocation)

Time Complexity (Start Operations): $\mathcal{O}(1)$ for both insert_at_start and delete_at_start, as they only require pointer reassignment at the head.

Time Complexity (End/Index Operations): $\mathcal{O}(n)$ in the worst case, as inserting or deleting at the end or a specific index requires traversing the list from the head to the target location.

Space Complexity: $\mathcal{O}(1)$ auxiliary space per operation (no extra arrays or recursive stacks are created during insertions or deletions). Overall structure takes $\mathcal{O}(n)$ space to store the nodes.

b)Write a Program to implement Double Linked List:

CODE:

```
1. class Node:
2.     def __init__(self, data):
3.         self.data = data
4.         self.next = None
5.         self.prev = None
6.
7. class DoublyLinkedList:
8.     def __init__(self):
9.         self.head = None
10.
11.     def insert_at_start(self, data):
12.         new_node = Node(data)
13.         if self.head is None:
14.             self.head = new_node
15.             return
16.         new_node.next = self.head
17.         self.head.prev = new_node
18.         self.head = new_node
19.
20.     def insert_at_end(self, data):
21.         new_node = Node(data)
22.         if not self.head:
23.             self.head = new_node
24.             return
25.         last = self.head
26.         while last.next:
27.             last = last.next
28.         last.next = new_node
29.         new_node.prev = last
30.
31.     def insert_at_index(self, index, data):
32.         if index == 0:
33.             self.insert_at_start(data)
34.             return
35.
36.         new_node = Node(data)
37.         current = self.head
38.
39.         for _ in range(index - 1):
40.             if not current:
41.                 raise IndexError("Index out of bounds")
42.             current = current.next
43.
44.         if not current:
45.             raise IndexError("Index out of bounds")
46.
47.         new_node.next = current.next
48.         new_node.prev = current
49.
50.         if current.next:
51.             current.next.prev = new_node
52.         current.next = new_node
53.
54.     def delete_at_start(self):
55.         if not self.head:
56.             print("List is empty.")
57.             return
58.         if not self.head.next:
59.             self.head = None
60.             return
61.         self.head = self.head.next
62.         self.head.prev = None
63.
64.     def delete_at_end(self):
65.         if not self.head:
66.             print("List is empty.")
67.             return
68.         if not self.head.next:
```

```

69.         self.head = None
70.         return
71.
72.         last = self.head
73.         while last.next:
74.             last = last.next
75.         last.prev.next = None
76.
77.     def delete_at_index(self, index):
78.         if not self.head:
79.             print("List is empty.")
80.             return
81.         if index == 0:
82.             self.delete_at_start()
83.             return
84.
85.         current = self.head
86.         for _ in range(index):
87.             if not current:
88.                 raise IndexError("Index out of bounds")
89.             current = current.next
90.
91.         if not current:
92.             raise IndexError("Index out of bounds")
93.
94.         if current.next:
95.             current.next.prev = current.prev
96.         if current.prev:
97.             current.prev.next = current.next
98.
99.     def display(self):
100.         elements = []
101.         current = self.head
102.         while current:
103.             elements.append(str(current.data))
104.             current = current.next
105.         print(" <-> ".join(elements) if elements else "Empty List")
106.
107.
108. if __name__ == "__main__":
109.     dll = DoublyLinkedList()
110.
111.     dll.insert_at_start(10)
112.     dll.insert_at_end(30)
113.     dll.insert_at_index(1, 20)
114.     dll.display()
115.
116.     dll.delete_at_start()
117.     dll.display()
118.
119.     dll.delete_at_end()
120.     dll.display()
121.
122.     dll.insert_at_start(10)
123.     dll.insert_at_end(30)
124.     dll.delete_at_index(1)
125.     dll.display()
126.

```

OUTPUT:

```
10 <-> 20 <-> 30
20 <-> 30
20
10 <-> 30
PS C:\Users\vamsi\Documents\College\SEM-3\DSA>
```

EXPLANATION:

Data Structure Type: Linear, Dynamic, Bidirectional

Time Complexity (Start Operations): $\mathcal{O}(1)$ for insert_at_start and delete_at_start.

Time Complexity (End/Index Operations): $\mathcal{O}(n)$ in the worst case for traversal. (*Note: Operations at the end could be reduced to $\mathcal{O}(1)$ if a tail pointer is added to the class.*)

Space Complexity: $\mathcal{O}(1)$ auxiliary space per operation.

Memory Overhead: Higher than a singly linked list. Each node requires extra memory to store the prev reference, leading to a slightly larger memory footprint.

ERROR TABLE:

SNO	ERROR	SOLUTION
1.	Indentation SyntaxError (Formatting Artifact)	The provided code block uses invisible non-breaking spaces instead of standard spaces for indentation. Highlighting and replacing all indents with standard keyboard spaces or tabs will allow the code to run perfectly.
2.	Algorithm Logic	No Errors Occurred. The insertion, deletion, and display methods all correctly handle doubly linked list pointers and standard edge cases (e.g., empty lists, ends of lists).

c) Write a python programme to implement Circular linked list:

CODE:

```
1. class Node:
2.     def __init__(self, data):
3.         self.data = data
4.         self.next = None
5.
6. class CircularLinkedList:
7.     def __init__(self):
8.         self.head = None
9.
10.    def insert_at_start(self, data):
11.        new_node = Node(data)
12.        if not self.head:
13.            self.head = new_node
14.            new_node.next = self.head
15.        return
16.
17.        temp = self.head
18.        while temp.next != self.head:
19.            temp = temp.next
20.
21.        new_node.next = self.head
22.        temp.next = new_node
23.        self.head = new_node
24.
25.    def insert_at_end(self, data):
26.        new_node = Node(data)
27.        if not self.head:
28.            self.head = new_node
29.            new_node.next = self.head
30.        return
31.
32.        temp = self.head
33.        while temp.next != self.head:
34.            temp = temp.next
35.
36.        temp.next = new_node
37.        new_node.next = self.head
38.
39.    def insert_at_index(self, index, data):
40.        if index == 0:
41.            self.insert_at_start(data)
42.        return
43.
44.        new_node = Node(data)
45.        temp = self.head
46.        count = 0
47.
48.        while temp and count < index - 1:
49.            temp = temp.next
50.            count += 1
51.        if temp == self.head:
52.            print("Index out of bounds")
53.        return
54.
55.        new_node.next = temp.next
56.        temp.next = new_node
57.
58.    def delete_at_start(self):
59.        if not self.head:
60.            print("List is empty")
61.        return
62.
63.        if self.head.next == self.head:
64.            self.head = None
65.        return
66.
67.        temp = self.head
68.        while temp.next != self.head:
```

```

69.         temp = temp.next
70.
71.         temp.next = self.head.next
72.         self.head = self.head.next
73.
74.     def delete_at_end(self):
75.         if not self.head:
76.             print("List is empty")
77.             return
78.
79.         if self.head.next == self.head:
80.             self.head = None
81.             return
82.
83.         temp = self.head
84.         while temp.next.next != self.head:
85.             temp = temp.next
86.
87.         temp.next = self.head
88.
89.     def delete_at_index(self, index):
90.         if not self.head:
91.             print("List is empty")
92.             return
93.
94.         if index == 0:
95.             self.delete_at_start()
96.             return
97.
98.         temp = self.head
99.         count = 0
100.
101.         while temp and count < index - 1:
102.             temp = temp.next
103.             count += 1
104.         if temp.next == self.head:
105.             print("Index out of bounds")
106.             return
107.
108.         temp.next = temp.next.next
109.
110.     def display(self):
111.         if not self.head:
112.             print("List is empty")
113.             return
114.         temp = self.head
115.         while True:
116.             print(temp.data, end=" -> ")
117.             temp = temp.next
118.             if temp == self.head:
119.                 break
120.         print(f"(back to head: {self.head.data})")
121.
122. if __name__ == "__main__":
123.     cll = CircularLinkedList()
124.
125.     cll.insert_at_end(10)
126.     cll.insert_at_end(20)
127.     cll.insert_at_start(5)
128.     cll.insert_at_index(2, 15)
129.     cll.display()
130.
131.     cll.delete_at_start()
132.     cll.delete_at_end()
133.     cll.display()
134.
135.     cll.delete_at_index(1)
136.     cll.display()
137.

```

OUTPUT:

```
5 -> 10 -> 15 -> 20 -> (back to head: 5)
10 -> 15 -> (back to head: 10)
10 -> (back to head: 10)
PS C:\Users\vamsi\Documents\College\SEM-3\DSA>
```

EXPLANATION:

Time Complexity (Start Operations): $\mathcal{O}(n)$ for both `insert_at_start` and `delete_at_start` in this specific implementation. Because the list is circular and we only track the head pointer, we have to traverse the entire list to find the very last node to update its next pointer. (*Note: You can easily drop this to $\mathcal{O}(1)$ if you modify the code to maintain a tail pointer alongside the head!*)

Time Complexity (End/Index Operations): $\mathcal{O}(n)$ in the worst case, as inserting or deleting at the end or a specific index requires traversing the list from the head to the target location.

Space Complexity: $\mathcal{O}(1)$ auxiliary space per operation (no extra arrays or recursive stacks are created during insertions or deletions). The overall structure takes $\mathcal{O}(n)$ space to store the actual nodes.

WEEK-6

a)Write a Program to implement Stack using Array:

CODE

```
1. class Stack:
2.     def __init__(self):
3.         self.stack = []
4.
5.     def push(self, item):
6.         self.stack.append(item)
7.         print(f"Pushed {item} to stack")
8.
9.     def pop(self):
10.        if self.is_empty():
11.            print("Stack Underflow")
12.        else:
13.            print(f"Popped {self.stack.pop()} from stack")
14.
15.    def peek(self):
16.        if self.is_empty():
17.            print("Stack is empty")
18.        else:
19.            print(f"Top element is {self.stack[-1]}")
20.
21.    def is_empty(self):
22.        return len(self.stack) == 0
23.
24.    def display(self):
25.        if self.is_empty():
26.            print("Stack is empty")
27.        else:
28.            print("Stack elements:", self.stack[::-1])
29.
30.    def menu(self):
31.        while True:
32.            print("\n1. Push")
33.            print("2. Pop")
34.            print("3. Peek")
35.            print("4. Display")
36.            print("5. Exit")
37.
38.            choice = input("Enter your choice (1-5): ")
39.
40.            if choice == '1':
41.                item = input("Enter item to push: ")
42.                self.push(item)
43.            elif choice == '2':
44.                self.pop()
45.            elif choice == '3':
46.                self.peak()
47.            elif choice == '4':
48.                self.display()
49.            elif choice == '5':
50.                print("Exiting...")
51.                break
52.            else:
53.                print("Invalid choice. Please try again.")
54.
55. my_stack = Stack()
56. my_stack.menu()
57.
```

OUTPUT:

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 1
Enter item to push: 5
Pushed 5 to stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 1
Enter item to push: 3
Pushed 3 to stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 4
Stack elements: ['3', '5', '4']

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 5
Exiting...
```

EXPLANATION:

The provided code implements a Stack data structure using a standard Python list within a single class. It operates on the Last-In, First-Out (LIFO) principle, meaning the most recently added item is the first one removed. The class bundles essential operations like push (adding items), pop (removing items), and peek (viewing the top item), along with an interactive menu method. By creating a Stack object and calling this menu, users can manage the stack directly from the command line in a continuous loop.

b)Write python program to implement stack using LinkedList

CODE:

```
1. class Node:
2.     def __init__(self, data):
3.         self.data = data
4.         self.next = None
5.
6. class Stack:
7.     def __init__(self):
8.         self.top = None
9.
10.    def push(self, data):
11.        new_node = Node(data)
12.        new_node.next = self.top
13.        self.top = new_node
14.        print(f'{data} is added to stack')
15.
16.    def pop(self):
17.        if self.top is None:
18.            print("Stack underflow")
19.        else:
20.            print(f'{self.top.data} is deleted from stack')
21.            self.top = self.top.next
22.
23.    def peek(self):
24.        if self.top is None:
25.            print("No elements in the stack")
26.        else:
27.            print(self.top.data)
28.
29.    def display(self):
30.        if self.top is None:
31.            print("Empty Stack")
32.        else:
33.            current = self.top
34.            while current:
35.                print(current.data, end="->")
36.                current = current.next
37.            print(None)
38.
39. s = Stack()
40. while (True):
41.     print("1.Push")
42.     print("2.Pop")
43.     print("3.Peek")
44.     print("4.Display")
45.     print("5.Exit")
46.     i = int(input("Enter your choice: "))
47.     if (i == 1):
48.         e = int(input("Enter element to add: "))
49.         s.push(e)
50.     elif (i == 2):
51.         s.pop()
52.     elif (i == 3):
53.         s.peek()
54.     elif (i == 4):
55.         s.display()
56.     elif (i == 5):
57.         exit()
58.     else:
59.         print("Choose correct choice")
60.
```

OUTPUT:

```

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add: 6
6 is added to stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add: 8
8 is added to stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 4
8->6->5->2->None
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 2
8 is deleted from stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 4
6->5->2->None

```

EXPLANATION:

push(data): Creates a new node, points its next reference to the current top node, and updates the top reference to point to this new node.

pop(): Checks for an empty stack (underflow). Reassigns the top pointer to the next node in the stack (top.next), allowing Python's garbage collector to remove the old top node.

peek(): Evaluates if the stack is empty. If not, it retrieves and outputs the data stored in the current top node without altering any node references.

display(): Traverses the stack starting from the top node, following the next pointers until it reaches the end (None), printing the data of each node along the way to represent the stack from top to bottom.

Data Structure Type: Linear, Dynamic (Non-contiguous memory allocation), operating on a Last-In, First-Out (LIFO) principle.

Time Complexity (Push, Pop, Peek Operations): $O(1)$ for push, pop, and peek, as they only require direct pointer reassignment or data access at the head (top) of the linked list without any traversal.

Time Complexity (Display Operation): $O(n)$ in the worst case, as displaying the full stack requires traversing every node from the top down to the bottom.

Space Complexity: $O(1)$ auxiliary space per operation (no extra arrays or recursive stacks are created during pushes or pops). The overall structure takes $O(n)$ space to dynamically store the nodes.

ERROR TABLE:

SNO	ERROR	SOLUTION
1.	AttributeError: 'NoneType' object has no attribute 'data'	Occurs if attempting to access <code>self.top.data</code> or <code>self.top.next</code> when the stack is empty. Ensure if <code>self.top</code> is <code>None</code> : (or <code>is_empty()</code>) is checked before trying to retrieve or modify the top node's attributes.
2.	Infinite Loop in display()	Caused by forgetting to advance the traversal pointer (e.g., omitting <code>current = current.next</code>) inside the while loop. Add the pointer update step to ensure the loop progresses and eventually terminates when <code>current</code> becomes <code>None</code> .